

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 5

1 - Enunciado

Implemente um programa em C que conte quantos números primos existem entre 2 e um número máximo n , depois, paralelize o laço principal usando a diretiva `#pragma omp parallel for` sem alternar a lógica original. compare o tempo de execução e os resultados das versões sequencial e paralela. Observe possíveis diferenças no resultado e no desempenho, e reflita sobre os desafios iniciais da programação paralela, como correção e distribuição de carga.

2 - Metodologia

Como descrito na tarefa foi criado duas versões de código para calcular os números primos, sendo uma versão executando de forma paralela e a outra na forma sequencial sem otimização na compilação.

```
gcc -O0 parallel.c -o parallel  
gcc -O0 sequence.c -o sequence
```

Foi utilizado a compilação sem otimização, pois o compilador ao executar o código de forma paralela estava sendo otimizado pelo compilador mesmo sem a utilização da flag durante a compilação, foi detectado a otimização ao fazer testes com níveis de otimização

diferentes, que não era algo requisitado pela tarefa, foi mais teste para ver o comportamento do código.

2.3 - Ambiente de Execução

Componente	Especificação
Notebook	Lenovo IdeaPad 3 15ALC6
SO	Ubuntu 26.04 LTS
Kernel	Linux 7.0.0-29-generic
CPU	AMD Ryzen 5 5500U — 6C/12T
Arquitetura	x86_64
RAM	9,05 GiB
Desktop	GNOME 50.1
Shell	Zsh 5.9
Terminal	Kitty 0.45.0
Compilador	GCC

3 - Código Fonte

3.1 - parallel.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define TAM 100000
```

```
int ehPrimo(int num){
    if(num < 2){
        return 0;
    }

    for (int i = 2; i < num ; i++){
        if (num % i == 0){
            return 0;
        }
    }

    return 1;
}

int main(){
    int count = 0;

    double inicio = omp_get_wtime();

    #pragma omp parallel for
    for(int i = 2; i < TAM; i++){
        if(ehPrimo(i)){
            count++;
        }
    }

    double fim = omp_get_wtime();

    printf("A quantidade de numeros Primos entre 2 e %d: %d\n", TAM, count);
    printf("Tempo Medido: %f (s) \n", fim - inicio);
}
```

```
    return 0;
}
```

3.2 - sequence.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define TAM 100000

static double medirTempo(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec * 1e-9;
}

int ehPrimo(int num){
    if(num < 2){
        return 0;
    }

    for (int i = 2; i < num ; i++){
        if (num % i == 0){
            return 0;
        }
    }

    return 1;
}
```

```
}

int main(){
    int count = 0;

    double inicio = medirTempo();
    for(int i = 2; i < TAM; i++){
        if(ehPrimo(i)){
            count++;
        }
    }
    double fim = medirTempo();

    printf("A quantidade de numeros Primos entre 2 e %d: %d\n", TAM, count);
    printf("Tempo Medido: %f (s) \n", fim - inicio);

    return 0;
}
```

4 - Resultado

Os testes dos programas em paralelo foram feitos utilizando de 1 até as 12 threads, sendo 12 o número máximo de threads disponíveis pelo hardware de teste como pode ser observado abaixo na figura 1.

```

OMP_NUM_THREADS=1 ./parallel
OMP_NUM_THREADS=2 ./parallel
OMP_NUM_THREADS=4 ./parallel
OMP_NUM_THREADS=6 ./parallel
OMP_NUM_THREADS=12 ./parallel
A quantidade de numeros Primos entre 2 e 100000: 9592
Tempo Medido: 2.752827 (s)
A quantidade de numeros Primos entre 2 e 100000: 9591
Tempo Medido: 2.027301 (s)
A quantidade de numeros Primos entre 2 e 100000: 9588
Tempo Medido: 1.165727 (s)
A quantidade de numeros Primos entre 2 e 100000: 9588
Tempo Medido: 0.860362 (s)
A quantidade de numeros Primos entre 2 e 100000: 9591
Tempo Medido: 0.618598 (s)

```

Figura 1: Teste parallel.c

Já para a execução do código sequencial não foi feita nenhum tipo de alteração ou otimização de código, foi feito da maneira mais convencional possível para ficar o mais próximo do real .

```

> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Par
> ./sequence
A quantidade de numeros Primos entre 2 e 100000: 9592
Tempo Medido: 2.758357 (s)

```

Figura 2: Teste sequence.c

Com os testes realizados foi montada a tabela abaixo:

Quantidade de threads	Quantidade de números Primos	Tempo Medido (s)	SpeedUp
1	9592	2.752827	1
2	9591	2.027301	1,360605554
4	9588	1.165727	2,366211815
6	9588	0.860362	3,20604234
12	9591	0.618598	4,459046101

Tabela 1: Resultados

5 - Conclusão

Os testes realizados foram extremamente interessantes para colocar em prática os conceitos de programação paralela. Começando com tempo de execução que varia com a quantidade de threads, quanto maior a quantidade de threads menor é o tempo medido o que quer dizer que temos um aumento de desempenho ao aumentar a quantidade de threads, porém esse aumento não é linear logo ter 12 threads ou ter 6 threads para esse caso não trás um aumento de desempenho dobrado.

Outro ponto interessante que pode ser observado no experimento é que apenas a execução com 1 thread traz o mesmo valor de números primos que na versão sequencial. Considerando que ambos os códigos estão corretos, isso significa que encontramos um problema de *race condition*, no qual, a partir da execução com 2 threads, a competição pelo acesso à variável compartilhada, que está sendo acessada e modificada concorrentemente pelas threads, acaba gerando resultados diferentes na quantidade de números primos.

Além da questão de correção, também é possível observar o desafio da distribuição de carga neste experimento. Por padrão, a diretiva `#pragma omp parallel for`, que divide o intervalo $[2, TAM)$ em blocos contíguos de tamanho igual entre as threads. Entretanto, o custo da função `ehPrimo(i)` não é constante, quanto maior o valor de i , mais divisores o laço interno precisa testar até encontrar um número primo. Isso significa que a thread responsável pelo último bloco (números próximos a TAM) realiza significativamente mais trabalho do que a thread responsável pelo primeiro bloco, gerando um desbalanceamento de carga que contribui para o speedup não-linear observado na Tabela 1.